Automation Tester Interview Prep Kit

Manual Testing (2 Years Experience) + Automation Testing (Selenium / TestNG / POM)

This set is organized to match a profile with hands-on manual testing experience (requirement analysis, test case design, defect management, API testing) and working-level automation testing skills (Selenium, TestNG, Page Object Model). Each question includes a concise model answer and a quick tip for how to personalize it with your own project examples.

1. Manual Testing — Core Concepts

Q1. What is the difference between Test Case, Test Scenario, and Test Script?

A test scenario is a high-level idea of what to test (e.g., 'verify login functionality'). A test case is a detailed, step-by-step document with preconditions, test steps, test data, and expected results derived from that scenario. A test script is the automated or scripted version of a test case, usually written in code for execution by a tool.

Tip: Mention that you wrote test cases in tools like JIRA/Excel with clear expected vs actual results.

Q2. Explain the STLC (Software Testing Life Cycle) phases.

STLC includes: Requirement Analysis, Test Planning, Test Case Design, Test Environment Setup, Test Execution, and Test Closure/Reporting. Each phase has entry and exit criteria that ensure quality gates are met before moving to the next phase.

Tip: Relate each phase to a real project — e.g., how you analyzed requirements for a feature like 'Get Quote'.

Q3. What is the difference between Verification and Validation?

Verification checks whether the product is being built correctly (review of documents, design, code — 'are we building the product right?'). Validation checks whether the built product meets user requirements (actual testing — 'are we building the right product?').

Q4. What is the difference between Smoke Testing and Sanity Testing?

Smoke testing is a shallow, wide check done on a new build to confirm the critical functionalities work before deeper testing begins. Sanity testing is a narrow, deep check done after a minor code change or bug fix to confirm that specific functionality works as expected.

Q5. What is Regression Testing, and how do you decide what to include in a regression suite?

Regression testing re-executes existing test cases to ensure new code changes haven't broken existing functionality. Regression suites are built by including test cases for core/critical flows, areas frequently affected by changes, and previously found high-severity defects.

Tip: Mention any regression suite you maintained or contributed to, and how often it was executed (per sprint/release).

Q6. How do you write an effective test case?

An effective test case has a unique ID, clear title, preconditions, numbered test steps, test data, expected result, and priority/severity. It should be atomic (tests one thing), clear enough for anyone to execute, and traceable back to a requirement.

Q7. What is the difference between Severity and Priority of a defect?

Severity indicates the impact of the defect on the application (e.g., critical, major, minor) from a technical standpoint. Priority indicates how urgently the defect should be fixed from a business standpoint. A defect can be high severity but low priority, or vice versa.

Tip: Give an example: a cosmetic issue on the homepage (low severity, high priority before a demo) vs. a rarely-used report crash (high severity, low priority).

Q8. Walk me through how you would report a bug you found.

I log it in the bug tracking tool (e.g., JIRA) with a clear title, steps to reproduce, expected vs. actual result, environment details, screenshots/logs, and severity/priority. I then track it through the workflow — new, assigned, fixed, retest, closed — and verify the fix before closing it.

Tip: Reference your actual tools — JIRA, Bugzilla, Trello — and mention how you retested fixes.

Q9. How do you perform requirement analysis before writing test cases?

I review requirement documents, user stories, or acceptance criteria, clarify ambiguities with BAs/developers, identify testable conditions, and prepare a requirement traceability matrix (RTM) to ensure every requirement maps to at least one test case.

Tip: Mention a real example — e.g., analyzing requirements for the 'Get Quote' page or HRMS module.

Q10. What is Boundary Value Analysis and Equivalence Partitioning? Give an example.

Equivalence Partitioning divides input data into valid and invalid partitions, assuming all values in a partition behave the same way, so only one representative value per partition needs testing. Boundary Value Analysis focuses on testing values at the edges of these partitions (min, min+1, max-1, max) since defects often occur at boundaries. Example: for an age field accepting 18–60, test 17, 18, 60, and 61.

Q11. What testing types have you performed in your projects?

Functional testing, regression testing, smoke/sanity testing, integration testing, UI testing, and API testing. I've also been involved in defining user stories and prioritizing backlog items in Agile sprints.

Tip: Tailor this list to your resume — mention specific modules like the logistics supplier module or instant quote feature.

Q12. How do you handle a situation where a developer disagrees that something is a bug?

I revisit the requirement or acceptance criteria to confirm expected behavior, provide clear reproduction steps and evidence (screenshots/logs), and if there's still disagreement, involve the BA/Product Owner to get clarity, keeping the discussion factual and requirement-driven rather than personal.

Tip: This is a behavioral question — use a real (brief) example if you have one.

2. API Testing (Postman)

Q13. What have you tested using Postman, and what do you check in an API response?

I've used Postman to validate backend functionality and frontend-backend integration — sending GET/POST/PUT/DELETE requests and verifying the HTTP status code, response body/schema, response time, and headers against expected values. I also test with valid, invalid, and edge-case inputs.

Tip: Reference a real example like validating the 'Get Quote' API or backend integration you tested.

Q14. What is the difference between GET and POST methods?

GET retrieves data from the server and sends parameters via the URL; it should not change server state and can be cached. POST sends data in the request body to create or update a resource on the server and is not typically cached.

Q15. What status codes are you familiar with, and what do they mean?

200 (OK) success, 201 (Created), 400 (Bad Request) — invalid input, 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), and 500 (Internal Server Error) — server-side failure.

Q16. How do you validate a JSON response in Postman?

I use the built-in test scripts (JavaScript-based) in the 'Tests' tab to assert status codes, response time, and JSON field values, e.g., pm.response.to.have.status(200) and pm.expect(jsonData.field).to.eql(expectedValue).

Tip: If you haven't written Postman test scripts yet, be honest and mention you've validated responses manually and are building scripting skills.

3. Automation Testing — Selenium, TestNG, POM

Q17. What is Selenium, and what are its main components?

Selenium is an open-source tool for automating web browsers. Its main components are Selenium WebDriver (for browser automation via code), Selenium Grid (for running tests in parallel across machines/browsers), and Selenium IDE (record-and-playback tool).

Q18. What is the Page Object Model (POM), and why is it used?

POM is a design pattern where each web page is represented as a class, with its web elements as variables and page-specific actions as methods. It separates test logic from page structure, making tests more readable, reusable, and easier to maintain — if a locator changes, you only update it in one place (the page class) instead of every test.

Tip: If you built even a small POM-based framework or practice project, describe its structure briefly.

Q19. What locators are available in Selenium, and which do you prefer?

Common locators are ID, Name, Class Name, Tag Name, Link Text, Partial Link Text, CSS Selector, and XPath. ID is generally fastest and most reliable when available; CSS Selectors are preferred over XPath for performance, though XPath is useful for complex DOM traversal (e.g., text-based or relative element location).

Tip: Since your background includes XPath, mention a scenario where you used XPath to locate a dynamic element.

Q20. What is the difference between findElement() and findElements()?

findElement() returns a single WebElement matching the locator and throws NoSuchElementException if not found. findElements() returns a list of all matching WebElements and returns an empty list (not an exception) if none are found.

Q21. How do you handle synchronization issues in Selenium (waits)?

I use explicit waits (WebDriverWait with ExpectedConditions) to wait for a specific condition like element visibility or clickability, rather than Thread.sleep(), which is a hard-coded wait and inefficient. Implicit wait can also be set globally as a fallback, but I avoid mixing implicit and explicit waits in the same script.

Q22. What is TestNG, and why is it used over JUnit in many Selenium frameworks?

TestNG is a testing framework inspired by JUnit but with more features suited for automation, such as annotations (@Test, @BeforeMethod, @BeforeSuite, etc.), test prioritization, grouping, parallel execution, data-driven testing via @DataProvider, and detailed HTML reports via testng.xml.

Q23. Explain a few key TestNG annotations you have used.

@BeforeSuite/@AfterSuite run once before/after all tests in the suite; @BeforeClass/@AfterClass run once before/after all methods in a class; @BeforeMethod/@AfterMethod run before/after each @Test method; @Test marks the actual test method; and @DataProvider supplies multiple sets of test data to a single test method for data-driven testing.

Q24. How would you design an automation framework from scratch for a new project?

I'd structure it using POM for page classes, TestNG for test execution and assertions, a config file (e.g., properties or YAML) for environment/URL/browser settings, a utility layer for reusable actions (waits, screenshots, reporting), and a reporting tool for results. Test data would be externalized (Excel/JSON) for

data-driven scenarios, and the suite would be organized so smoke, regression, and full suites can run independently via testng.xml.

Tip: This is a strong signal question for a 'fresher in automation' — even a conceptual, well-structured answer works well if you haven't built a full framework yet.

Q25. What is the difference between hard assertion and soft assertion in TestNG?

A hard Assertion (Assert class) stops test execution immediately when a check fails. A Soft Assertion (SoftAssert class) records the failure but continues executing the remaining steps in the test, and all failures are reported together when assertAll() is called at the end.

Q26. How would you handle a dynamic web element (e.g., an ID that changes on every page load)?

I'd use a locator strategy that avoids the dynamic part — such as CSS/XPath with contains(), starts-with(), or a stable attribute (name, class, or a nearby static label) — instead of relying on the changing ID directly.

Q27. What is the difference between Selenium WebDriver and Selenium Grid?

WebDriver automates and controls a single browser instance directly on one machine. Selenium Grid allows distributing and running tests in parallel across multiple machines, browsers, and OS combinations, using a hub-node architecture (or the newer Grid 4 router-based setup).

Q28. Since you're moving from manual to automation testing, how do you decide what to automate first?

I'd prioritize stable, frequently executed, and high-business-value test cases — like smoke and core regression flows — over one-off or UI-heavy exploratory tests. Automation candidates should have predictable steps, clear pass/fail criteria, and not change frequently, since unstable UI leads to high script maintenance.

Tip: This question tests judgment, not just tool knowledge — a thoughtful answer matters more than naming every tool.

4. Behavioral / Experience-Based Questions

Q29. Tell me about a challenging bug you found and how you handled it.

Use the STAR method: Situation (context of the feature/module), Task (what you were testing), Action (how you identified, isolated, and reported the bug — including any edge cases you checked), and Result (impact — e.g., prevented a production issue, improved a key feature).

Tip: Pick a real example from a module you tested, like the 'Get Quote' page or an HRMS feature.

Q30. How do you prioritize your testing tasks when time is limited before a release?

I focus first on critical and high-priority business flows, then regression around recently changed areas, using risk-based testing to decide where limited time gives the most coverage and confidence.

Q31. Why are you moving from manual testing into automation testing?

I want to build on my strong manual testing foundation — requirement analysis, test design, and defect management — by adding automation skills (Selenium, TestNG, POM) to increase test coverage, speed up regression cycles, and grow technically alongside the QA field's direction toward automation-first testing.

Tip: Keep this forward-looking and confident — don't frame it as still 'learning'; frame it as actively building on existing QA strength.

Q32. How do you work within an Agile team as a tester?

I participate in sprint planning and backlog grooming to clarify requirements early, write test cases against user stories and acceptance criteria, test incrementally as features are developed, and report status/blockers in daily stand-ups, ensuring testing keeps pace with the sprint.

Tip: Mention your experience defining user stories and prioritizing the backlog if relevant.

5. Quick-Reference Cheat Sheet

Concept	One-line Definition
STLC	Requirement Analysis -> Planning -> Test Design -> Environment Setup -> Execution -> Closure
Smoke vs Sanity	Smoke = wide & shallow on new build; Sanity = narrow & deep after a fix
Severity vs Priority	Severity = technical impact; Priority = business urgency to fix
POM	Design pattern separating page structure (locators) from test logic
Explicit Wait	Waits for a specific condition (e.g., element clickable) before proceeding
TestNG @DataProvider	Supplies multiple data sets to a single test method (data-driven testing)
Hard vs Soft Assertion	Hard stops on failure; Soft collects all failures, reports at the end
Equivalence Partitioning	Group inputs into valid/invalid classes; test one value per class
Boundary Value Analysis	Test edge values (min, min+1, max-1, max) where defects commonly occur

Preparation tip: For every answer above, keep one real project example ready (module name, what you tested, and the outcome) so you can back up definitions with practical evidence during the interview.